

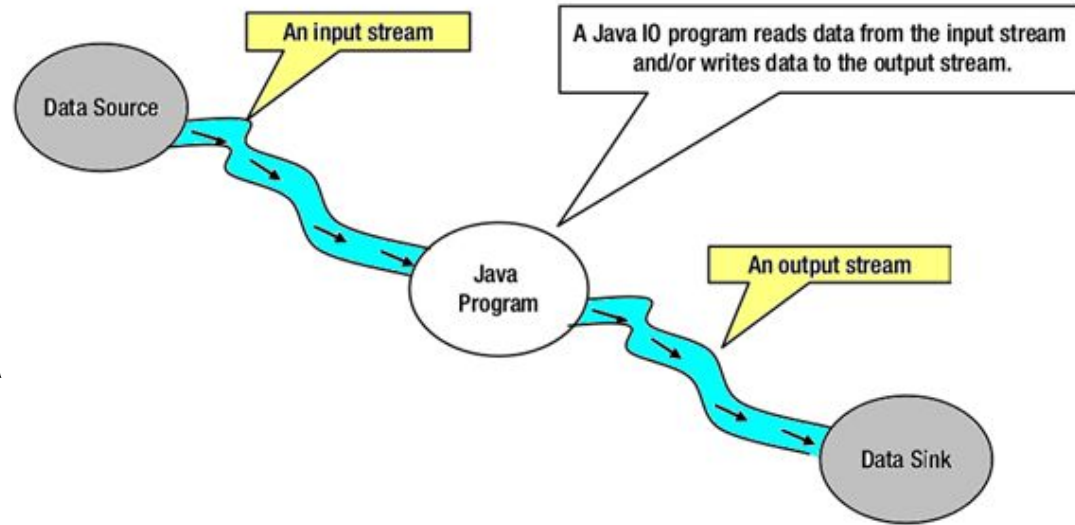
# Advanced Programming

## CSE 201

Lecture 15: I/O Streams | Serialization & Deserialization

# I/O Streams

- Stream is a sequence of data
  - Flows in/out the program to/from an external source such as file, network, console, etc.
- Similar to a stream of flowing water...
- Program uses **input stream** to read data from a source, one at a time
- Program uses **output stream** to write data to a destination, one at a time



# Types of Streams

- Two types of streams
  - Byte stream
  - Character stream
- Byte stream
  - Operates upon stream of “byte” ASCII code (8-bit)
- Character stream
  - Operates upon stream of “character” Unicode (16-bit)

# java.io Package

- Reading

open a stream

while more information

**read** information

close the stream

- Writing

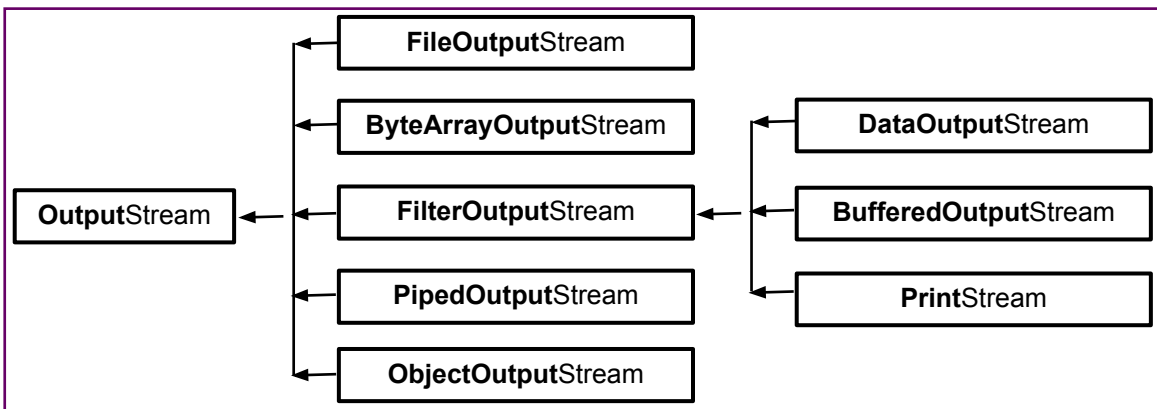
open a stream

while more information

**write** information

close the stream

# Byte Stream Hierarchy

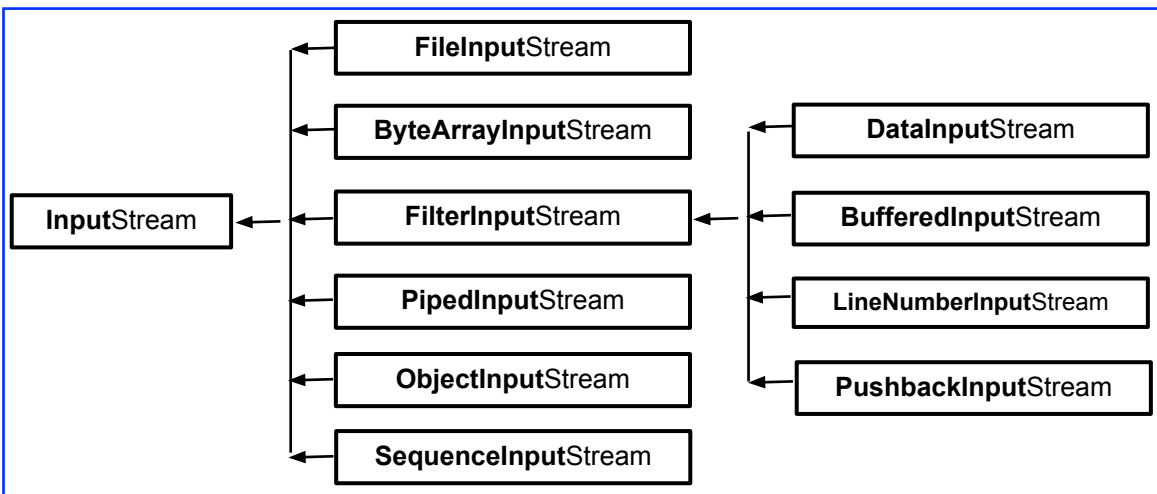


## ● OutputStream

- This is the abstract class
- Parent class of all classes representing an output stream of bytes

## ● InputStream

- This is the abstract class
- Parent class of all classes representing an input stream of bytes



# Byte Streams in System Class

```
public final class System {  
    public static final InputStream in;  
    public static final PrintStream out;  
    public static final PrintStream err;  
    .....  
}
```

```
import java.io.*;  
  
class TakeByte {  
    public static void main(String args[]) throws IOException{  
        int c=System.in.read();  
        System.out.print((char)c);  
    }  
}
```

- In java, 3 streams are created for us automatically. All these streams are attached with console
  - **System.out**: standard output stream
  - **System.in**: standard input stream
  - **System.err**: standard error stream

→ Typecasting is necessary because c contains the ASCII value

# Byte Stream Example

```
import java.io.*;

class Main{
public static void main (String args[]) throws IOException
{
    FileInputStream in = null;
    FileOutputStream out = null;
    try {

        in = new FileInputStream("input.txt");
        out = new FileOutputStream("output.txt");
        int c;
        c = in.read();//reads next byte, and returns -1 if the end of stream is reached
        while (c!=-1) {

            out.write(c);
            c = in.read();

        }
    } finally {
        if (in != null)
            in.close();
        if (out != null)
            out.close();
    }
}
```

- **InputStream**

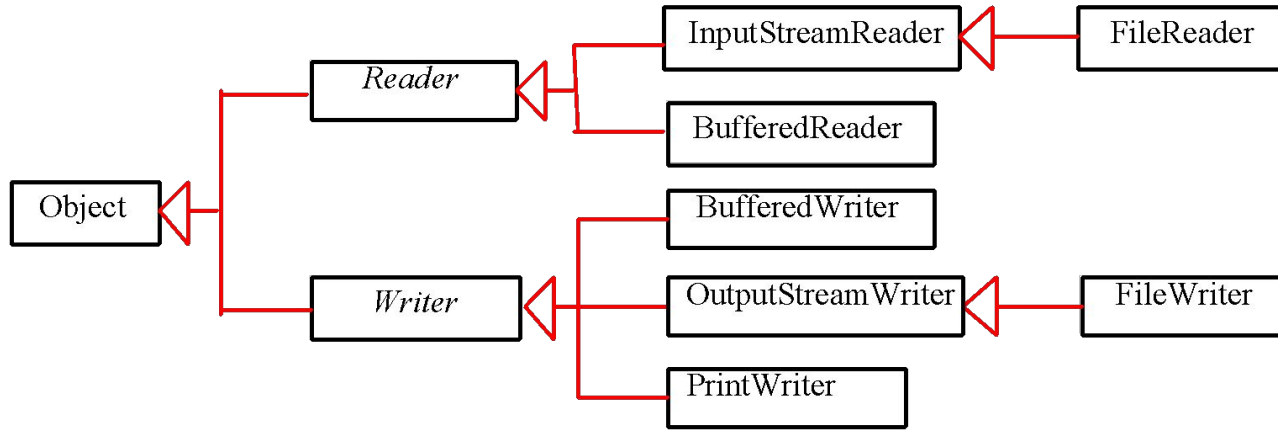
- **read()** – read the next byte of data from the input stream
- **close()** – close input stream

- **OutputStream**

- **write(int)** – write a byte to current output stream
- **close()** – close output stream

- **Byte stream is used for low-level I/O, e.g., processing binary files**

# Character Stream Hierarchy



- All character stream classes are subclasses of Reader and Writer class
- Used for processing text files (character by character)



# Character Stream Example

```
public static void main(String args[])
    throws IOException
{
    FileReader in = null;
    FileWriter out = null;
    try {
        // both constr. throws FileNotFoundException
        in = new FileReader("input.txt");
        // throws IOException
        out = new FileWriter("output.txt");
        int c;
        c = in.read();
        while (c != -1) {
            out.write(c);
            c = in.read();
        }
    }finally {
        if (in != null)
            in.close();
        if (out != null)
            out.close();
    }
}
```

- This example is very similar to the byte stream I/O
- In terms of coding, the difference is in using `FileReader` and `FileWriter` for input and output
- Note that “int” type variable is used in both these examples to read and write. Although internally they are working differently:
  - In byte stream example, the “int” variable holds a 8 bit value (ASCII Code)
  - In this example, the “int” variable holds a 16 bit value (Unicode)

# Buffered Streams (1/2)

```
public static void main(String args[])
    throws IOException
{
    BufferedReader in = null;
    PrintWriter out = null;
    try {
        in = new BufferedReader( new
            FileReader("input.txt"));
        out = new PrintWriter( new
            FileWriter("output.txt"));
        String l;
        while ((l = in.readLine()) != null){
            out.println(l);
        }
    }finally {
        if (in != null)
            in.close();
        if (out != null)
            out.close();
    }
}
```

- Combine streams into chains to achieve more advanced input and output operations
- Reading/Writing character by character is slow
- Faster to read/write a larger block of data, such as a line.
- Helps read the entire line.
- readLine function returns the next line. It returns null if there are no more lines.

# Using Scanner Class

```
public static void main(String args[]) {  
    Scanner in = new Scanner(System.in); //java.lang  
    // Scanner class implements iterator  
    while (in.hasNext()) {  
        System.out.println(in.next());  
    }  
    in.close();  
}
```

hasNext() returns true if this scanner has another token in its input.

next() finds and returns the next complete token from this scanner and returns it as a string; a token is usually ended by whitespace such as a blank or line break. If no token exists, NoSuchElementException is thrown.

# Buffered Streams (2/2)

```
public static void main(String args[])
    throws IOException
{
    Scanner in = null;
    PrintWriter out = null;
    try {
        in = new Scanner( new BufferedReader( new
            FileReader("input.txt")));
        out = new PrintWriter( new
            FileWriter("output.txt"));
        while (in.hasNext()) {
            out.println(in.next());
        }
    }finally {
        if (in != null)
            in.close();
        if (out != null)
            out.close();
    }
}
```

- Here we are combining three classes for breaking input into tokens:
  - Scanner
  - BufferedReader
  - FileReader
- BufferedReader will read one line at a time
- Scanner will be able to parse this line by white space separated tokens

# Serialization and Deserialization



- Serialization in Java is a mechanism of writing the state of an **object** into a **byte stream**
- The reverse operation is called deserialization
- Some usage
  - Storing live objects in a file
  - Hibernating applications
  - Moving object state over the network

# java.io.Serializable Interface

```
package java.io;  
public interface Serializable { /*empty*/ }
```

- Must be implemented by the class to be serialized
- This is a tag/marker interface similar to Cloneable interface

# Example: Serializing and Deserializing

```
1. import java.io.*;
2.
3. class Manager implements Serializable {
4.     private String name;
5.     public Manager(String n) { ..... }
6. }

7. public class Main {
8.     public static void serialize()
9.         throws IOException {
10.         Manager s1 = new Manager("Amy");
11.         ObjectOutputStream out = null;
12.         try {
13.             out = new ObjectOutputStream (
14.                 new FileOutputStream("out.txt"));
15.             out.writeObject(s1);
16.         } finally {
17.             out.close();
18.         }
19.     }
20. }

/* Continued on RHS window */
```

```
/* Continued from LHS window */
21. public static void deserialize()
22.     throws IOException, ClassNotFoundException {
23.     ObjectInputStream in = null;
24.     try {
25.         in = new ObjectInputStream (
26.             new FileInputStream("out.txt"));
27.         Manager s1 = (Manager) in.readObject();
28.     } finally {
29.         in.close();
30.     }
31. }

32.
33. public static void main(String[] args)
34.     throws IOException, ClassNotFoundException {
35.     serialize();
36.     deserialize();
37. }
38.
39. } /* End of Main class */
```

```

import java.io.*;
class Address {
    private String city;
    public Address(String c) { ..... }
}
class Manager implements Serializable {
    private String name;
    private Address addr;
    public Manager(String n, String city) { ..... }
}
public class Main {
    public static void serialize() throws IOException {
        Manager s1 = new Manager("Amy", "Delhi");
        ObjectOutputStream out = null;
        try {
            out = new ObjectOutputStream (
                new FileOutputStream("out.txt"));
            out.writeObject(s1);
        } finally {
            out.close();
        }
    }
    .....
}

```

- This program compiles fine but will generate **NotSerializableException**
- All fields of Manager class should either be primitive type or serializable objects
  - Address class should also implement Serializable interface



- There is no point in serializing static field members in a class
  - Static fields do not represent object state but they represent class state
  - There will not be any compilation/runtime issue, although the value serialized will not make any sense as it can always be updated later in the class.
- **transient** keyword in Java
  - If you don't want any field to be serialized then mark that as "transient".
  - When JVM comes across **transient** keyword, it ignores original value of the variable and save default value of that variable data type.

```
import java.io.*;
class Test implements Serializable
{
    // Normal variables
    int i = 10, j = 20;

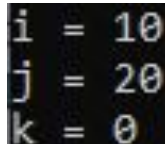
    // Transient variables
    transient int k = 30;
}

class Main{

    public static void main(String[] args) throws Exception
    {
        Test input = new Test();

        // serialization
        FileOutputStream fos = new FileOutputStream("output.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(input);

        // de-serialization
        FileInputStream fis = new FileInputStream("output.txt");
        ObjectInputStream ois = new ObjectInputStream(fis);
        Test output = (Test)ois.readObject();
        System.out.println("i = " + output.i);
        System.out.println("j = " + output.j);
        System.out.println("k = " + output.k);
    }
}
```



```
i = 10
j = 20
k = 0
```